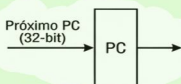


Para que a arquitetura RISC-V saia do papel e se torne algo funcional, é necessário o uso correto de seus componentes. São eles: Banco de registradores, unidade de controle, ULA, memória, entrada/saída, PC, somador, geração de valor imediato e extensão de sinal.

PROGRAM COUNTER (PC)



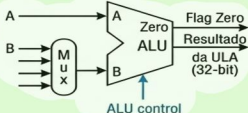
Armazena o endereço da instrução atual.

MEMÓRIA DE INSTRUÇÕES



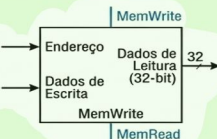
Busca a instrução baseada no PC.

UNIDADE LÓGICA E ARITMÉTICA (ULA/ALU)



Executa operações aritméticas e lógicas.

MEMÓRIA DE DADOS



Armazena e recupera dados da memória.

BANCO DE REGISTRADORES (REGISTERS)



Armazena 32 registradores de uso geral.

GERADOR DE IMEDIATOS (IMM GEN)



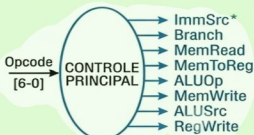
Estende os campos de imediato para 32 bits.

CONTROLE DA ULA



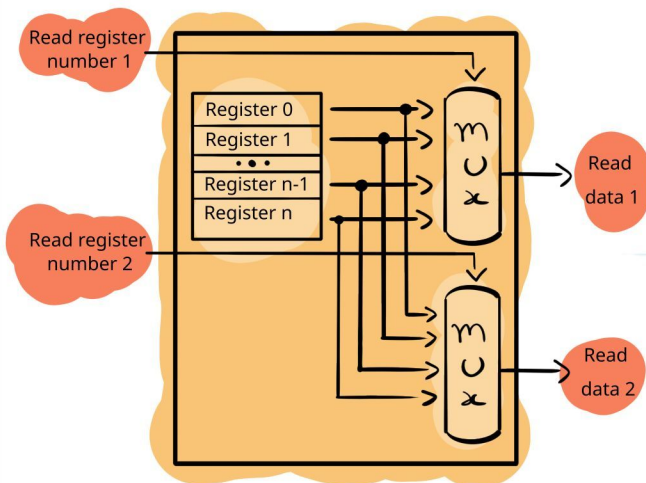
Decodifica a operação específica da ULA.

CONTROLE PRINCIPAL

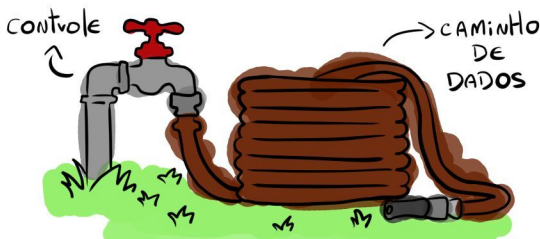


Gera sinais de controle principais para o datapath.

Olhando mais atentamente para o banco de registradores podemos observar que ele possui multiplexadores internos usados para filtrar os valores que serão usados. Na imagem abaixo abordamos apenas a operação de leitura de registradores para simplificar.

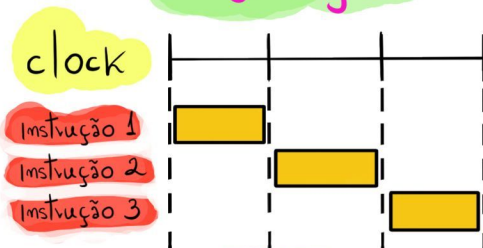


Todos esses componentes precisam ser interconectados. Para que esse objetivo se torne viável criou-se a distinção entre linhas de controle e linhas de dados. Nessa dinâmica, o caminho de dados pode ser representado pela mangueira de jardim, enquanto que o controle pode ser representado pela torneira.



Esses componentes podem ser unidos de formas diferentes para a execução de instruções. Na RISC-V temos 3 implementações clássicas: Single-cycle, multi-cycle e pipelined.

single-cycle



multi-cycle



Pipelined



Independente da abordagem, o conjunto básico de instruções se mantém

tipo R

add
sub
and
or
slt

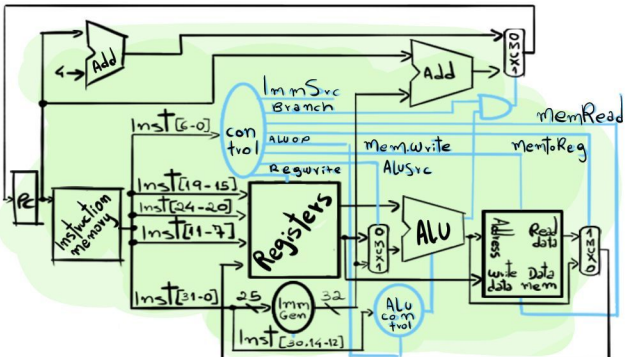
tipo load/store

lw
sw

tipo desvio

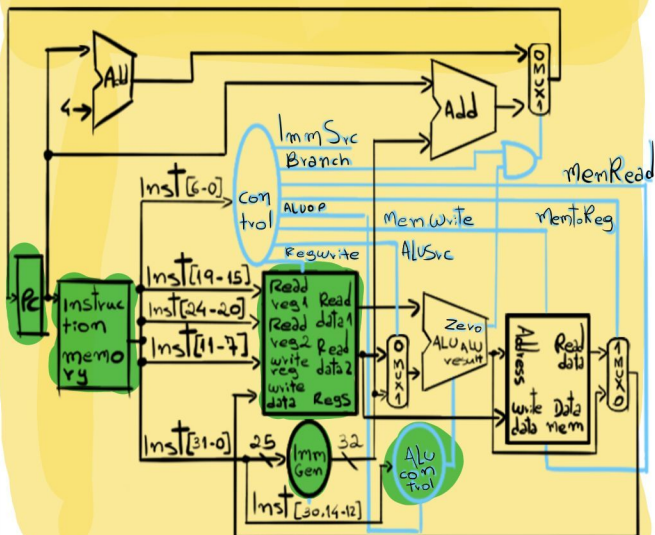
beq

Um exemplo de todos esses elementos trabalhando juntos pode ser observado no diagrama da abordagem single-cycle. O diagrama demonstra o caminho percorrido pelos dados e como cada instrução é movida para que a execução seja feita da maneira correta.

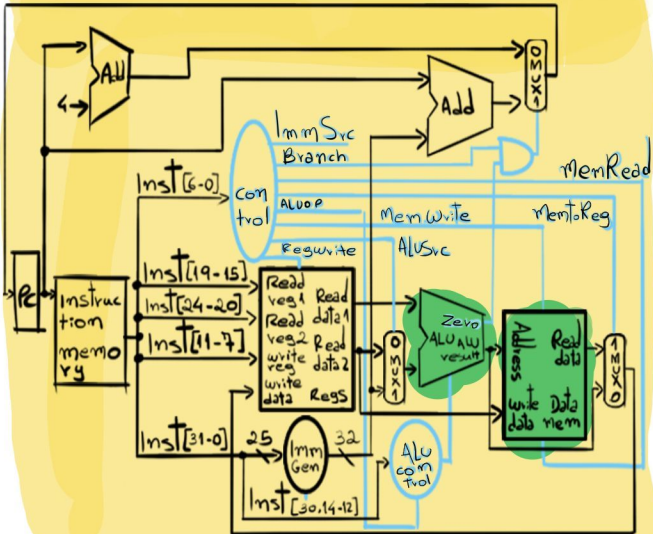


Para entender como o caminho de dados funciona, podemos olhar para a instrução lw. Primeiro o PC envia o endereço da instrução atual para a instruction memory, e esse componente cospe a instrução para fora, continuando o caminho. A instrução é quebrada de acordo com o seu formato (essa é a etapa de decodificação) e os bits são enviados para o bloco de registradores, control e Imm Gen.

caminho da instrução lw



Aqui a ALU (ULA) entra em ação. Ela recebe dois dados: O primeiro vem sempre do Read data 1 e o segundo do Imm Gen (Se for um Load/Store), filtrado pelo MUX. Em seguida, o caminho vai em direção ao Data Memory, que usa o resultado da ULA como endereço. Por fim, o dado que estava guardado sai em read data.



Ao final do ciclo, O último Mux (na direita) decide que o que será gravado no registrador de destino é um dado que veio do Data Memory. Assim, esse valor entra no Write data e é salvo permanentemente no banco de registradores. Tudo isso acontece em um único ciclo de clock!